

ChargeShield

Cost-sensitive fraud scoring and bounded chargeback defence

The model proposes. A deterministic policy engine decides.

Track	Razorpay AI Buildathon 2026 — Track 02, AI Risk Manager
Posture	Defence-only. The system never blocks a transaction.
Dataset	IEEE-CIS Fraud Detection (public). Not Razorpay data.
Held-out	88,581 transactions, final 15%, chronological
Headline	Test PR-AUC 0.543 vs 0.223 constrained baseline
Dispute metric	Wasted representment 37.3% vs 48.7% contest-all
Tests	214 cases · 162 behavioural · 0 asserting nothing
Reproducibility	make all — 83s from a clean clone, bit-exact
Guide generated	2026-09-05 · commit a24c31f

Contents

1	The problem, in the merchant's terms
2	The idea, and the one thesis it defends
3	Architecture
4	The repository, directory by directory
5	Data pipeline and leakage control
6	Two models, and why the baseline is deliberately weak
7	Calibration — and the metric that flattered us
8	The cost model, and the result that said we were unnecessary
9	Evaluation and error analysis
10	The dispute spine
11	The policy engine — seven rules in order
12	The LLM layer
13	API and interface
14	Testing strategy
15	Results, stated honestly
16	Decisions log
17	Failure log
18	Limitations and claim discipline
19	What the panel will ask
20	Command reference

1 The problem, in the merchant's terms

A chargeback is a customer disputing a completed payment with their bank. The merchant has already shipped the goods and already recognised the revenue. They now have a deadline, a pile of documents, and three bad options.

Option	What it costs
Accept every dispute	Forfeit the amount on cases you would have won.
Contest every dispute	Pay representment cost on cases that were genuinely fraudulent — you lose those anyway, so the ops spend is pure waste.
Review everything by hand	Analyst time on every dispute, most of which are obvious in either direction.

Nobody tells the merchant **which disputes are worth fighting**. That is the gap. It is a triage problem with real money attached, not a classification problem in search of an application.

Note the polarity carefully, because it is inverted from every fraud model you have seen. A fraud screen asks *should I block this?* and blocks when risk is high. ChargeShield asks *should I fight this chargeback?* — and a **high** probability of genuine fraud means **do not fight it**, because you will lose and pay for the privilege. Low fraud probability means the transaction was legitimate and therefore defensible.

2 The idea, and the one thesis it defends

ChargeShield scores fraud risk on a temporally held-out public benchmark, calibrates that score into a real probability, converts the probability into a rupee-denominated decision using a stated cost model, and then puts every decision through a deterministic policy engine that an LLM cannot influence.

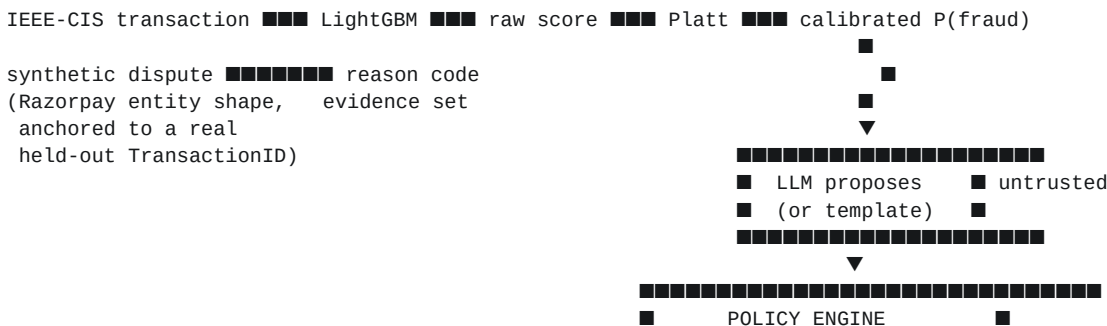
The thesis: an LLM is useful for language and untrustworthy for authority. It may draft a representment. It may not decide to file one. Everything in the architecture follows from taking that seriously.

Concretely, that means the LLM's output is a *proposal* — a validated data structure with no privileged status — and the component that turns proposals into actions is a pure function with no network client, no filesystem handle and no source of randomness. It cannot call anything. That is not a promise in a README; it is enforced by a test that deletes `socket` and `open` out from under it and asserts the gate still works.

The second thesis: claim discipline

A hackathon project can claim anything. This one is built so a reviewer can check. Every headline number is regenerated by `make all` from a clean clone in 83 seconds and matches to six decimal places. Every assumption that cannot be measured is named as an assumption and swept. Where the honest result is unflattering — and it is, in section 8 — the unflattering number is reported first.

3 Architecture



4 The repository, directory by directory

Path	What it is	Why it exists
ml/	Offline pipeline	Everything that learns. Never imported by the gate.
ml/data_prep.py	Load, join, downcast, split	Chronological split on TransactionDT. Overlap asserted, not trusted.
ml/features.py	Feature partition	Explicit lists, not prefix matching. Asserts at import that every starter column is classified exactly once.
ml/train_baseline.py	Logistic regression	A deliberate floor. One sklearn Pipeline is the leakage control.
ml/train_lightgbm.py	Primary model	392 features. One CategoricalDtype learned on train.
ml/calibrators.py	Platt / isotonic / identity	Separate module so pickles load outside __main__. See Failure 06.
ml/calibrate.py	Calibration selection	Fits on val_fit, chooses on val_pick.
ml/cost_curve.py	Rupee cost model	Threshold and bands, swept across scenarios.
ml/evaluate.py	Held-out evaluation	Detection metrics, policy comparison, error slices.
ml/link_disputes.py	The spine	Anchors synthetic disputes to real transactions.
ml/metrics.py	Shared scoring	Both models scored by identical code, on purpose.
ml/predict.py	Serving path	Partial feature vectors; NaN is a valid input.
app/policy/	The gate	Pure functions. No I/O, no LLM, no clock, no randomness.
app/policy/thresholds.py	Cost-derived bands	The only I/O in the package.
app/policy/evidence_policy.py	Sufficiency + fabrication	Two checks that must not be conflated.
app/policy/action_policy.py	decide()	Seven ordered overrides. Returns the whole chain.
app/services/	Wiring	Everything with a side effect.
.../template_response.py	Deterministic proposal	Written before any LLM call. Cannot fabricate.
.../llm_service.py	The only provider-aware module	Validates shape, never repairs content.
.../audit_service.py	Append-only JSONL	One record per decision.
.../batch_runner.py	Queue-wide run	Prices four policies with the real gate.
.../risk_service.py	Model wrapper	The only place the API touches the model.
app/api/	FastAPI routers	No endpoint returns a bare proposal.
app/models/schemas.py	Pydantic contracts	DisputeProposal: extra=forbid, frozen.
frontend/index.html	One screen	A decision record. Relative URLs only.
tests/	214 cases	Including 43 adversarial.
scripts/	Operator tools	check_llm, test_census, build_guide.
demo/run_demo.py	Recorded demo	Self-asserting. Exits non-zero on drift.
config/	costs.yaml, split.yaml	Every assumption that moves a number.

Path	What it is	Why it exists
evidence/requirements.json	Reason code to evidence	Field names from Razorpay docs, cited.
evaluation/	Committed metrics	The numbers the README quotes.
artifacts/	Models + metadata	JSON committed, .pkl regenerable.

The separation that matters: **ml/ imports nothing from app/policy at runtime, and app/policy imports nothing from ml/ at all.** The policy package is stdlib plus yaml. That is what keeps the gate auditable in one sitting — a reviewer can read three files and know what can happen.

One deliberate exception: `ml/cost_curve.py` imports the `indifference-threshold` function from `app/policy/thresholds.py`. The policy engine is the authority on the decision boundary; the cost analysis is a study of it. Two copies of that formula is how the analysis and the engine silently come to disagree.

5 Data pipeline and leakage control

IEEE-CIS ships 590,540 transactions with a 3.5% fraud rate, plus a separate identity table that matches only 24.4% of them. The split is **chronological on TransactionDT** — 70 / 15 / 15 — because fraud is non-stationary and a random split would let the model learn from the future. All three splits come from `train_transaction.csv`; the competition's test file has no labels.

```
train = df[dt < train_end]          # < on the lower bound
val   = df[(dt >= train_end) & (dt < val_end)]  # >= on the upper
test  = df[dt >= val_end]

assert train.TransactionDT.max() < val.TransactionDT.min()
assert val.TransactionDT.max() < test.TransactionDT.min()
assert not (train_ids & val_ids) and not (val_ids & test_ids)
```

The asymmetric comparison is the point: `<` on one side and `>=` on the other means no transaction can land in two splits. It is asserted in the pipeline and again in `tests/test_split.py`.

A finding worth having ready

Missingness in this dataset is **not stationary across the split**. M1 is 53.8% missing in train and 26.8% in val; `id_01` goes 73.4% to 82.3% the other way. So the missing-indicator features carry genuine train/val distribution shift. That is a property of time-splitting real data, not a bug — but it is a reason not to over-read a val/test gap, and it is stated in the code rather than discovered by a reviewer.

How leakage is actually prevented

Structurally, not by discipline. For the linear model, one sklearn Pipeline is the sole path from raw row to matrix: `.fit` is called exactly once, on train, and val and test only ever reach `.predict_proba`. There is no code path that can fit on held-out data. Two tests give that teeth:

- The fitted imputer medians equal the **train** medians — and pooling train+val demonstrably changes them, so the test can actually detect pooling. A leakage test that cannot fail is not a test.
- Transforming a 100-row slice is bitwise identical to those same rows inside the full frame. That is the strongest available proof that no cross-row statistic is computed at transform time.

6 Two models, and why the baseline is deliberately weak

The baseline is not a weak attempt. It is a constrained model, and the gap between it and the tree measures what the constraint cost.

	Logistic regression	LightGBM
Input columns	46	392
Features after encoding	97	392
High-cardinality IDs	excluded	included
V-columns	excluded	all 339
Missing values	median + indicator	native NaN direction
Categoricals	one-hot	native splits
Test PR-AUC	0.223	0.543
Test ROC-AUC	0.820	0.901

The exclusion, and how to defend it

We use a constrained starter feature set, one-hot encode the manageable low-cardinality categoricals, and deliberately exclude high-cardinality identifier fields from the first linear baseline to control dimensionality and avoid unnecessary complexity.

Seven columns are excluded: card1 (12,242 levels), DeviceInfo (1,546), card2 (500), addr1 (318), card5 (110), card3 (105), addr2 (67). One-hotting card1 alone adds ~12,000 columns fit on noise. And these are *anonymised codes* — their numeric magnitude is meaningless, so passing them as floats would make a linear model interpolate between identifiers. Neither option is defensible, so they are cut and the tree gets them.

The evidence that this was the right call: four of the tree's top eight features by gain — DeviceInfo, card2, card1, addr1 — are exactly the columns the baseline excludes. The prediction was tested, not asserted.

Three encoding decisions specific to linearity

- **log1p on TransactionAmt.** It spans 0.25 to 31,937 with a median of 68.95. A raw linear term claims each extra dollar shifts the log-odds by a constant, which is the wrong functional form for a log-normal amount.
- **StandardScaler is not cosmetic.** L2 penalises coefficient magnitude, so with unscaled features the penalty is applied in arbitrary units and shrinks whichever feature happens to be measured large.
- **drop=None, never drop='first'.** An unseen category encodes as all-zeros; under drop='first' that is indistinguishable from the dropped reference level, silently relabelling unknowns as the reference category. Keeping every level gives 'unknown' its own signature.

The LightGBM trap

Calling `.astype("category")` separately per split makes pandas assign integer codes *per frame, in order of appearance*. 'visa' could be code 3 in train and code 1 in val. LightGBM stores a split as "code in {1,3}", so the served model reads scrambled categories — no exception, no warning, just quietly worse predictions. One CategoricalDtype is learned on train and applied unchanged, and a test verifies the naive version genuinely differs on this data.

7 Calibration — and the metric that flattered us

The cost model computes expected loss as $p \times \text{amount}$. That arithmetic is meaningless unless p is a real probability. A model can rank perfectly and still be badly calibrated, so this is verified rather than assumed.

Candidates — uncalibrated, Platt, isotonic — are fit on **val_fit** (first 70% of val, chronological) and the winner is chosen by Brier on **val_pick** (the last 30%, unseen by both). Fitting and choosing on the same slice would always pick isotonic: it has far more freedom and would be partly fitting noise. That is a formality, not a comparison.

The number that was wrong

Platt won and reported an aggregate ECE of 0.0050, down from 0.0077. That looked like a clean result. Then the reliability plot disagreed with it, and the score distribution explained why: **92% of test rows score below 0.05**. An aggregate ECE is therefore almost entirely a measurement of the region where no decision is ever made.

On test, $p \geq 0.10$ (where the gate acts)	n	ECE	expected frauds	actual	bias
Uncalibrated	4,011	0.0512	1,644	1,762	-6.7%
Platt (selected)	5,748	0.0478	2,244	1,969	+14.0%

Calibration in the decision region is roughly ten times worse than the headline. The number this project quotes is 0.048, not 0.0050.

Platt was kept anyway, on portfolio grounds: summed across all transactions the uncalibrated model expects 2,489 frauds against an actual 3,083 — under-predicting total fraud by 19.3%, which distorts summed expected loss far more than a 14% over-count confined to 5,748 rows. Platt lands at +4.8%. The residual bias also errs toward ACCEPT, the conservative direction for a defence-only product.

A design detail worth mentioning if asked about metric choice: signed relative bias is reported alongside unsigned ECE, because expected loss is a *sum* and a consistent-direction error does not cancel across a portfolio. ECE scores +0.05 and -0.05 identically; for this application they are not the same thing.

Platt is strictly monotonic, so it **cannot** reorder — ROC-AUC must be identical before and after. That is asserted in code, and it is the kind of self-check that catches an implementation error rather than a modelling one.

8 The cost model, and the result that said we were unnecessary

ChargeShield does not block transactions, so the two errors are not 'blocked a good customer' and 'missed a fraud'. They are:

Action	Truth	Expected cost
ACCEPT	either	A — you eat the chargeback
CONTEST	legitimate	$c + (1-w)A$ — pay the cost, usually recover
CONTEST	real fraud	$c + A$ — pay the cost <i>and</i> still lose

where c is the representment cost and w the assumed success rate on a legitimate transaction. Note c is paid in both CONTEST rows — representment costs the same whether or not it works. That is the entire reason a threshold exists.

The threshold is analytically amount-dependent

Indifference sits at $c = w \cdot A \cdot (1-p)$, so $p^*(A) = 1 - c/(w \cdot A)$. A small dispute is never worth contesting; a large one is worth contesting even at appreciable fraud risk. This is why the policy engine has an amount cap at all — the cap is cost-derived, not an arbitrary safety rail.

Dispute amount	p^*	Review band	Width
Rs 2,000	0.643	[0.536, 0.750]	0.214
Rs 6,070 (median)	0.882	[0.847, 0.918]	0.071
Rs 20,000	0.964	[0.954, 0.975]	0.021

Failure 03: the first answer was 'contest everything'

The first working sweep returned a cost-optimal policy of contesting 99% of disputes, beating the trivial rule by about 0.01%. Read literally: the model adds nothing.

The arithmetic was correct and the answer was meaningless — which is the most dangerous combination, because nothing fails and no test goes red. The population was wrong. Pricing every held-out transaction as a dispute gives a 3.5% fraud queue, and at that base rate contesting is positive-EV almost regardless of score.

The fix was a prior-shift sensitivity: re-calibrate the scores on the odds scale and re-weight the population to a range of assumed dispute-queue fraud rates. That answers the question a reviewer actually asks — *at what queue composition does this earn its keep?*

Assumed queue fraud rate	t^*	Share contested	Rs saved/dispute vs contest-all
3.5% (as observed — artefactual)	0.890	99.0%	2
20%	0.895	91.8%	27
35%	0.910	84.2%	57
50%	0.910	74.5%	93
65%	0.885	59.8%	137

Both the embarrassing result and the informative ones are reported. Deleting the first and keeping the second would have been the easy move and the dishonest one.

9 Evaluation and error analysis

Each model is scored at **its own** val-selected threshold. A shared threshold would be wrong: the baseline's `class_weight='balanced'` deliberately rescales its scores off LightGBM's axis. Scored at LightGBM's 0.25 it reads precision 0.056 / recall 0.917 — a meaningless comparison that flatters recall and buries precision. PR-AUC and ROC-AUC are threshold-free and *are* comparable.

The top failure mode, and the confound underneath it

Slicing FP/FN by amount band, ProductCD and identity-presence produced two apparent findings: the model fails on ProductCD 'W' (86% of unrecovered fraud value) and it fails when identity data is missing (86%). Both true, both large — and **they are the same population**. Jaccard 0.994; every W transaction lacks identity data.

Recall	identity present	identity missing
ProductCD = W	n = 0	0.213 (n=69,468)
non-W	0.703 (n=18,668)	0.704 (n=445)

The 2x2 separates them. Non-W transactions that *also* lack identity still recall at 0.704 — identical to when identity is present. **Missing identity is not the driver; ProductCD W is**. The obvious reading was confounded and would have put a confidently wrong root cause in the README. Caveat stated honestly: the disambiguating cell holds only 27 frauds, so this is suggestive rather than conclusive.

Failure modes are ranked by **share of unrecovered fraud value**, not by recall. A slice with terrible recall on 40 transactions is a curiosity; the failure mode worth naming is the one carrying the money.

10 The dispute spine

This is where the two halves meet. Each synthetic dispute is anchored to a real held-out TransactionID and carries that transaction's **real isFraud label** plus the model's calibrated score. That is what makes one dispute-side metric a measurement rather than a simulation.

How the queue is generated, and why not at random

A random sample of held-out transactions has a 3.5% fraud rate — section 8 showed that is degenerate. But sampling to hit the config's assumed 50% would be circular: constructing the population to match the assumption under test. So the queue comes from a process with two named parameters and the composition *falls out*:

```
P(dispute | fraud)      = 0.80    most fraud eventually gets charged back
P(dispute | legitimate) = 0.03    friendly fraud, non-receipt, confusion

-> 5,013 disputes, 48.65% fraud
```

That it lands near the config's 0.50 is **by construction, not by measurement**, and the artifact says so. A reviewer who disagrees argues with the two parameters rather than with the conclusion — which is the point of naming them.

What is real, documented, and constructed

Real	The anchor transaction, its isFraud label, its amount, the model's score.
Documented	Dispute entity field names, status and phase vocabularies, evidence field names — from Razorpay's API reference, cited with retrieval date.

Constructed	Which transactions get disputed, the reason-code vocabulary, the code-to-evidence mapping, which evidence is present, all identifiers.
-------------	--

Everything constructed lives under a `_chargeshield` namespace, never inside the Razorpay-shaped body, and a test asserts the body contains only documented fields. The claim is 'mirrors the documented dispute entity' — one smuggled field and that claim is false.

Correction made during the build: the original evidence requirements used `authentication_record`, `delivery_confirmation` and `service_access_log`. None of those are real Razorpay evidence fields. The real ones are `shipping_proof`, `billing_proof`, `proof_of_service`, `access_activity_log`, `cancellation_proof` and others. Rewritten from the actual docs, with the code-to-evidence *mapping* clearly marked as this project's construction, because Razorpay publishes no such matrix.

11 The policy engine — seven rules in order

`decide()` is a pure function. No I/O, no LLM, no clock, no randomness. It cannot reach the network because it has no client; it cannot read a threshold from disk because thresholds arrive as an argument. A reviewer can verify that by reading one file, and a test enforces it by monkeypatching `socket` and `open` out from under it.

#	Rule	Action	Why <i>this</i> position
1	<code>amount_cap_exceeded</code>	HUMAN_REVIEW	Exposure limit first — it holds even if the score is wrong.
2	<code>proposal_cited_evidence_not_on_file</code>	HUMAN_REVIEW + reject	Before sufficiency. Otherwise a fabricated citation slips through whenever the real evidence happens to be complete.
3	<code>dispute_too_small_to_repay_representation</code>	ACCEPT	$w \cdot A \leq c$: no score or evidence can change the answer, so escalating spends Rs 150 to learn nothing.
4	<code>required_evidence_missing</code>	HUMAN_REVIEW	Cannot substantiate a contest.
5	<code>inside_cost_review_band</code>	HUMAN_REVIEW	Automating is worth less than asking.
6	<code>fraud_probability_above_band</code>	ACCEPT	Contesting likely-genuine fraud burns cost to lose.
7	<code>proposal_honoured</code>	CONTEST	Cost-minimising, evidence on file.

Rules 2 and 3 are the two orderings a panel should probe, and both have answers. Fabrication before sufficiency, because a proposal that invents evidence is untrustworthy about everything, not just this dispute. The economic floor before the evidence gate, because a dispute that cannot repay a representment gets accepted regardless — escalating it buys nothing.

The review band was wrong once, and the fix is the interesting part

The first version came out [0.595, 1.000] — 0.405 wide, swallowing the ACCEPT action entirely. The error was the *level* of comparison: it measured where one global policy's total cost sits near the optimum, which is a statement about policies, not about a dispute. Per dispute:

$$E[\text{cost} \mid \text{CONTEST}] - E[\text{cost} \mid \text{ACCEPT}] = c - w \cdot A \cdot (1-p)$$

$$\text{escalate when } |\text{that}| < h \rightarrow 1 - (c+h)/(w \cdot A) < p < 1 - (c-h)/(w \cdot A)$$

That is ~0.07 wide at the median amount, and it moves with the amount. The band width is set by what a human costs: a cheap analyst widens it, an expensive one narrows it. **It was not tuned to look good** — adjusting `human_review_cost` until the band was comfortable would be reverse-engineering, not derivation.

This also corrected the architecture. The original decision table sent $p \sim 0.47$ to HUMAN_REVIEW as an 'uncertainty band'. That is wrong: at $p=0.47$ on a Rs 6,070 dispute contesting is clearly correct and there is nothing to adjudicate. What warrants a human is proximity to the **cost indifference point**, not to $p=0.5$. Probability uncertainty and decision uncertainty are different quantities. The table was changed to follow the arithmetic.

Every decision reports the whole chain

`decide()` returns not just the rule that fired but every rule with its outcome — fired, passed (and why it did not fire), or not reached. A verdict says what happened; the chain says what was examined. An auditor asking 'was the amount cap checked?' should not have to infer it. Recording is separate from deciding, so control flow is unchanged.

12 The LLM layer

The **deterministic template was written before any LLM call**. That ordering is the design: the LLM is an enhancement to a system that already works, not a dependency of it. `propose()` never raises — no key, no network, a timeout, malformed JSON, or two schema violations all end at the template.

The template **cannot** fabricate evidence, structurally: `cited_evidence` is derived from the evidence object, so no code path produces a citation for a document that does not exist. That makes it the safe floor — an LLM proposal can only be worse on that axis, which is why the gate checks fabrication on every proposal regardless of source.

Four decisions worth defending

- **p_fraud is not sent to the model**. The LLM does language, the classifier does probability, the policy does deciding. Sending the score would let the model launder it into a decision and blur three roles into one.
- **llm_service validates shape, never repairs content**. It does not strip fabricated citations. If it did, the gate would never fire and there would be no audit record showing the model tried.
- **One retry, then the template**. A provider returning garbage twice is one to stop calling, not to keep paying.
- **Provider exceptions are logged by type, not message**. Error text can carry request context; the type is also what distinguishes a timeout from an auth failure. The message-level diagnostic lives in a hand-run script instead.

Credentials

`LLM_API_KEY` from the process environment, with `OPENAI_API_KEY` as a fallback. **No module in this project reads `.env`**. The key is never logged, printed, placed in an exception message, written to the audit log, or returned. A test raises a provider error *containing the key* and asserts it never reaches the log; another asserts neither the key nor its first or last eight characters appear in `/health`.

Provider portability

`LLM_BASE_URL` and `LLM_MODEL` swap providers with no code change — OpenAI, Groq and Google AI Studio all speak the same wire format, so one client plus a base URL covers all three. A class per provider would add indirection without capability.

JSON mode is the one call parameter that varies between them, so a 400 naming `response_format` triggers one retry without it; a 429 or 401 deliberately does not, because dropping JSON mode would not help and would muddy the diagnosis.

Verified live

Working path: Groq, `openai/gpt-oss-120b`, ~1.2-1.8s per call. Three probes, all schema-valid, gate correct on each. **Failure path:** OpenAI with an exhausted account — three real 429s (`credit_balance_exhausted`), two retries each, three clean degradations to the template.

*The model did not fabricate. The second probe is a deliberate temptation — `billing_proof` present, `shipping_proof` absent — and it correctly reported `INSUFFICIENT` rather than inventing a delivery record. So every fabrication case in the demo and the dashboard is a **constructed** proposal, labelled as such. The gate exists because a model may fabricate, not because this one did.*

13 API and interface

Endpoint	Purpose
GET /health	Artifacts loaded; credential <i>presence</i> only, never any part of the value.
POST /score	Calibrated p_fraud from a partial feature vector, plus amount-dependent bands.
POST /disputes/analyze	Proposal (LLM or template) → policy decision → audit record.
POST /disputes/draft	Representment draft, returned <i>inside</i> the decision. Never submits.
POST /disputes/validate	Gate a caller-supplied proposal. No LLM. The demo endpoint.
POST /batch/run	Re-run the gate over the dispute queue.
GET /audit/{id}	The record for a decision.
GET /metrics	Committed evaluation artifacts, for the interface.

No endpoint returns a bare proposal. /disputes/draft returns the draft inside the decision — handing back ungated LLM output would let a caller act on it and defeat the entire design.

/score accepts **partial** feature vectors because no caller has all 392. LightGBM learns a NaN direction at every split, so an absent feature is handled exactly as in training, where 76% of rows had no identity data. The response reports features_supplied, because a prediction from four features is not the same evidence as one from four hundred and the caller should be able to see which they got.

The interface is one page, no framework, styled as a numbered decision record rather than a dashboard. Section 2 issues a live POST /disputes/validate per scenario and renders the real seven-rule chain; sections 1 and 3 read committed artifacts. The colophon states which is which, so a reader never has to guess whether a figure was computed for display. It uses relative URLs only, so it works wherever the API runs.

14 Testing strategy

Group	Files	What it defends
Data & leakage	test_split, test_preprocessing, test_lightgbm_encoding	Temporal integrity, fit-on-train-only, category mapping stability
Model & cost	test_calibration, test_cost_curve	Monotonicity, ECE correctness, the loss function hand-checked on paper
Dispute spine	test_disputes	Anchoring, schema conformance, claim limits
Policy gate	test_policy, test_audit	Purity, precedence, auditability
LLM boundary	test_schema, test_failure_modes	Contract, fallback, secret handling
Adversarial	test_adversarial_llm	43 cases attacking the gate
API	test_api	No ungated output, no credential leakage
Demo	test_demo	The recording cannot drift from the system
Meta	test_artifacts, test_suite_integrity	The repo about itself

Counting honestly

A raw pytest count flatters a suite, so make census breaks it down and **fails the build if any test file or function asserts nothing**:

Behavioural test functions	162
----------------------------	-----

Meta / hygiene functions	7
Empty scaffold files	0
Functions asserting nothing	0
Cases pytest collects	214

The census found three bare tests. One was a **fixture named test_split** in test_disputes.py — which reads as a test case to anyone skimming the file. Its own first version reported eighteen, a false positive from substring-matching ast.dump() where pytest.raises renders as attr='raises'. Both are in the failure log.

Adversarial tests, and what they are not

43 cases attack the gate from four directions: proposals lying about evidence (single and multiple fabrications, invented evidence types, citing a field that is present but empty, claiming SUFFICIENT when it is not), prompt injection inside free text, malformed structured output (ten shapes), and provider failure (six kinds).

Every proposal in that file is CONSTRUCTED. None was produced by a language model, and none is counted anywhere as an observed hallucination. These are engineering tests of a defence — the distinction between 'we tested for this' and 'this happened' is one the project makes deliberately.

One property is asserted across every attack shape: **no bad proposal ever reaches CONTEST**. Injection payloads are inert by construction, because the gate never reads reasoning_summary or draft_representment — it acts on the evidence object, the amount and the score.

15 Results, stated honestly

Detection, held-out final 15%

Model	PR-AUC	ROC-AUC	Threshold	Precision	Recall	F1
Logistic regression	0.223	0.820	0.79	0.213	0.361	0.267
LightGBM (Platt)	0.543	0.901	0.25	0.552	0.498	0.523

Prevalence is 3.5%, so PR-AUC 0.543 is roughly 15x the base rate. Against the project's own red flag — PR-AUC > 0.90 on a temporal split means hunt for leakage — this is comfortably in the believable range.

The one dispute metric with real ground truth

Contested	1,803
...of which genuinely fraudulent (real isFraud)	672 → 37.3%
Contest-everything would waste	48.7%
Relative reduction	23.4%

Policy comparison — and the negative headline

Policy	Rs / dispute	vs contest-all	vs amount rule
Contest nothing	12,464	-3,631	-3,640
Contest everything	8,833	0	-8
Amount rule, no model	8,825	+8	0
ChargeShield	8,843	-9	-18

Across the whole queue ChargeShield costs 18 rupees per dispute MORE than the best no-model policy. That is the honest number and it is reported first. The aggregate hides two opposing effects:

Segment	n	ChargeShield vs amount rule
Evidence complete and under the cap	2,209	+69
Evidence complete	2,477	+49
Evidence incomplete	2,536	-83
All disputes	5,013	-18

Where the gate has what it needs, it wins. Where evidence is missing it loses, because it pays Rs 150 for a human rather than filing a representment it cannot substantiate. **That is a safety property bought deliberately, not a modelling failure.** Human review alone costs Rs 84 per dispute — more than the entire spread between all four policies.

And the share of disputes with incomplete evidence is a **parameter** (`p_required_evidence_present = 0.70`), not an observation. The net figure moves with a dial, not with the model.

16 Decisions log

Every cut, with the reason and the cost accepted. Read verbatim from DECISIONS.md so this table cannot drift from the repository.

Date	Decision	Reason	Cost accepted
Aug 30	SQLite over PostgreSQL	Single-process batch analytical workload. Postgres would be resume decoration.	No concurrent writes
Aug 30	Plain HTML over React	~2h vs ~1 day for the same screen. Metrics don't depend on the UI.	Less polished demo
Aug 30	No agent framework	Workflow is retrieve → classify → propose → validate → gate. A graph library adds indirection, not capability.	—
Sep 1	Exclude 7 high-cardinality ID columns (card1, DeviceInfo, card2, addr1, card5, card3, addr2) from the linear baseline	Anonymised codes: numeric magnitude is meaningless to a linear model, and one-hot on card1 alone is +12,242 columns fit on noise. 46 inputs -> 97 features instead of ~13,000.	Baseline understates achievable performance. Deliberate — it is a floor, and LightGBM splits on these natively
Sep 1	class_weight="balanced" on the baseline LR	At 3.5% prevalence an unweighted model at t=0.5 predicts ~nothing positive and reports recall ~0. Makes point metrics non-degenerate.	Destroys calibration. Baseline scores are rankings, not probabilities — calibration is a separate step
Sep 1	Threshold selected on val, applied unchanged to test	Selecting the operating point on test is a second, subtler leak than the split itself.	Test F1 is lower than a test-tuned number would be. That is the point
Sep 1	LightGBM gets the 339 V-columns and all 7 identifiers; the baseline keeps neither	Makes the two models a controlled contrast rather than an algorithm bake-off. The gap measures what the baseline's constraint cost.	Two feature sets to maintain; ml/metrics.py shared so scoring is identical
Sep 1	Native LightGBM categoricals via one train-fitted CategoricalDtype, not per-split .astype("category")	pandas assigns codes per frame in order of appearance; LightGBM stores splits as integer codes. Per-split casting silently scrambles categories at inference. Verified on this data — the mappings genuinely differ.	A learned artifact that must ship with the model
Sep 1	card1/2/3/5, addr1/2 numeric for the tree, not categorical	12,242 levels overfits LightGBM's categorical splitter; a tree can carve an arbitrary partition of a numeric axis given enough splits.	DeviceInfo (string, 1,546 levels) has no numeric fallback and stays categorical
Sep 1	No scale_pos_weight on LightGBM	Threshold comes from the val PR curve anyway, so re-weighting buys nothing and damages the probability scale ml/calibrate.py needs. The baseline needed class_weight only to make t=0.5 non-degenerate.	Point metrics at t=0.5 are degenerate; irrelevant, we don't use t=0.5
Sep 1	Calibrators fit on val_fit (first 70% of val), winner chosen on val_pick (last 30%)	Fitting Platt and isotonic on val and then picking by Brier on the same val always picks isotonic — it has more freedom and partly fits val's noise. That is a formality, not a comparison.	val now does three jobs (early stopping, calibrator fit, method selection) and is not a held-out estimate
Sep 1	Selection by Brier, not ECE	Brier is a proper scoring rule, so it rewards being correct <i>and</i> honest about uncertainty. ECE alone is gamed by a model that predicts the base rate for everything.	ECE still reported — it is the interpretable number

Date	Decision	Reason	Cost accepted
Sep 1	Kept Platt despite a near-tie	ECE 0.0077 → 0.0050 on test is a real halving of the calibration gap; Brier is a wash (0.02205 → 0.02214). Two parameters, cannot overfit.	Honest framing: calibration was worth <i>verifying</i> more than it was worth <i>applying</i>
Sep 1	Report calibration restricted to $p \geq 0.10$, not just aggregate	92% of rows score below 0.05, so aggregate ECE measures the region where no decision is made. Aggregate 0.0050 vs decision-region 0.0478 — a 10x difference, and the second one is the real one.	Reported metrics look worse. They are more true
Sep 1	Kept Platt despite +14% over-prediction inside the decision region	Uncalibrated under-predicts total fraud by 19.3% portfolio-wide, which distorts summed expected loss more than a 14% over-count on 5,748 rows. Over-prediction also errs toward ACCEPT — conservative for a defence-only product.	Decision-region miscalibration is a stated limitation of the cost curve, not a hidden one
Sep 1	Dispute-defence cost model, not screening	ChargeShield does not block transactions. Replaced <code>fp_cost_inr/fn_cost_inr</code> (block-vs-allow) with representment cost + forfeited amount. Polarity inverts: CONTEST at LOW $p(\text{fraud})$.	<code>config/costs.yaml</code> scenario keys rewritten
Sep 1	Forfeited-case cost is amount-weighted from TransactionAmt	Forfeiting a \$97,000 dispute is not the same as forfeiting a \$500 one, and it makes <code>auto_action_amount_cap_inr</code> cost-coherent rather than arbitrary.	Requires joining amounts back to the prediction files
Sep 1	<code>assumed_win_rate_if_legitimate</code> is an explicit swept parameter	Pricing "forfeited winnable case" is impossible without it, and real chargeback outcomes are not public. Better a stated assumption swept 0.50–0.85 than a silent 1.0.	The most attackable number in the project — labelled as such, never reported as measured
Sep 1	Production bands derived at an assumed 50% dispute-queue fraud rate, not the observed 3.5%	The 3.5% is an artefact of scoring every transaction as a dispute. Tuning the policy engine on a population the analysis itself calls artefactual would be the wrong kind of consistency.	Another stated assumption; full sensitivity reported in <code>cost_curve.json</code>
Sep 1	Did NOT tune <code>human_review_cost_inr</code> to narrow the review band	The band came out [0.595, 1.000] — wide, and it swallows ACCEPT. Adjusting the input to produce a comfortable output is reverse-engineering, not derivation.	Band width is an open problem for Day 3's policy engine, flagged rather than hidden
Sep 1	Compare against an amount-only static rule, not just defend-none/defend-all	The trivial baselines are easy to beat and flatter the model. The cost-optimal no-model policy is the real adversary, and it already captures most of defend-all's value.	The reported ML contribution drops from \$93 to \$84 per dispute. That is the honest number
Sep 1	Rank failure modes by share of unrecovered fraud VALUE, not by recall	A slice with terrible recall on 40 rows is a curiosity, not a failure mode. Value-weighting finds the one that costs money.	—
Sep 1	Added an explicit confound check to the error analysis	Slicing alone reported "identity missing" and "ProductCD W" as two findings; they are one population (Jaccard 0.994). The 2x2 shows W is the driver and identity missingness is not.	Without it the README would have stated a confidently wrong root cause
Sep 1	<code>tests/test_suite_integrity.py</code> names the three empty stub files	Docstring-only test files pass silently and inflate the suite count. Declaring them makes the gap visible and forces a deliberate deletion when they are written.	—

Date	Decision	Reason	Cost accepted
Sep 2	Disputes generated from $P(\text{dispute} \neq \text{fraud}) = 0.80 / P(\text{dispute} \neq \text{legit}) = 0.03$, not sampled to a target rate	A random sample gives a 3.5% queue, which <code>cost_curve.py</code> showed is degenerate. Sampling to hit 50% would be circular — building the population to match the assumption under test. Two named parameters let the composition fall out, and give a reviewer something concrete to argue with.	Queue lands at 48.65%, close to assumed <code>dispute_fraud_rate: 0.50</code> — by construction, and stated as such
Sep 2	Constructed data namespaced under <code>_chargesield</code> , never inside the Razorpay body	The claim is "mirrors Razorpay's documented dispute entity". One invented field sitting next to real ones makes that claim false. <code>tests/test_disputes.py</code> asserts the body contains only documented fields.	Slightly noisier JSON
Sep 2	<code>evidence/requirements.json</code> rewritten from the actual Razorpay docs	The placeholder used <code>authentication_record</code> , <code>delivery_confirmation</code> , <code>service_access_log</code> — none of which are real Razorpay evidence fields. Real ones are <code>shipping_proof</code> , <code>billing_proof</code> , <code>proof_of_service</code> , <code>access_activity_log</code> , etc. Cited with retrieval date.	The reason-code → <code>evidence mapping</code> is still constructed, and is labelled as constructed inside the file
Sep 2	Review band derived per dispute , not globally	The global band was $[0.595, 1.000]$ — 0.405 wide, and it swallowed ACCEPT entirely. The error was comparing <i>policy</i> costs, not <i>per-dispute</i> action costs. Per dispute: $1 - (c+h)/(w \cdot A) < p < 1 - (c-h)/(w \cdot A)$, ~0.07 wide at the median.	The band moves with amount, so there is no single number to quote — a table is required
Sep 2	<code>docs/ARCHITECTURE.md</code> decision table corrected to match the cost model	It sent $p \approx 0.47$ to HUMAN_REVIEW on "uncertainty". Wrong: at $p=0.47$ on a ₹6,070 dispute contesting is clearly correct. What warrants a human is proximity to the <i>cost indifference point</i> ($p^*=0.882$), not to $p=0.5$.	The architecture diagram changed to follow the arithmetic, not the reverse
Sep 2	Fabrication checked before evidence sufficiency	With sufficiency first, a fabricated citation slips through whenever the real evidence happens to be complete. A proposal that invents evidence is untrustworthy about everything, not just this dispute.	—
Sep 2	Economic-floor rule ($w \cdot A \leq c \rightarrow$ ACCEPT) placed before the evidence gate	A dispute that cannot repay a representment is accepted regardless of score or evidence, so escalating it spends ₹150 of analyst time to learn nothing.	The evidence gate does not fire on futile disputes — deliberate
Sep 2	<code>decide()</code> is keyword-only	<code>p_fraud</code> and <code>amount_inr</code> are both floats; a positional swap would produce a plausible, completely wrong decision that no type checker would catch.	Slightly more verbose call sites
Sep 2	Audit log is JSONL, not SQLite	Append-only, greppable, no migrations, diffable on camera. Each record fingerprints the cost assumptions so it stays explainable after <code>costs.yaml</code> changes.	<code>db_models.py</code> SQLite remains the Tier-2 path if querying is needed
Sep 2	Price the real gate on the dispute queue , not on transactions	The gate needs <code>reason_code</code> and evidence; transactions have neither. <code>ml/evaluate.py</code> structurally cannot run it, which is why it was silently pricing something else.	Two comparison tables to explain instead of one — but only one of them is the product

Date	Decision	Reason	Cost accepted
Sep 2	Report the -18 net before the +69 actionable-segment figure	The net is the honest headline. Leading with the favourable segment would repeat the exact error of Failure 04 in a subtler form.	The headline number is negative
Sep 2	Human review modelled as a <i>non-oracle</i> analyst	Assuming analysts decide perfectly would flatter every comparator except ours (we are the policy that routes to humans), so the optimistic assumption is the conservative one for our claim.	ChargeShield's measured cost is higher than an oracle model would show
Sep 2	<code>p*(A)</code> has one implementation, in <code>app/policy/thresholds.py</code>	It was implemented twice — the policy engine and the cost analysis could silently disagree about the decision boundary. Same argument that put scoring in <code>ml/metrics.py</code> .	<code>ml/</code> now imports from <code>app/policy/</code> ; the policy package stays <code>stdlib</code> -only so this stays cheap
Sep 3	<code>DisputeProposal</code> gains a <code>cited_evidence</code> field	The original sketch could not support the fabrication check: to know whether a proposal invented a 3-D Secure record you need what it <i>claimed</i> to rely on. Without it the "LLM proposes CONTEST → policy BLOCKS" demo is undemonstrable.	One more field for the model to get wrong — which is the point
Sep 3	<code>p_fraud</code> is not sent to the LLM	The model does language, the classifier does probability, the policy does deciding. Sending the score would let the LLM launder it into a decision and blur the three roles.	The LLM assesses evidence without knowing the fraud risk — deliberate
Sep 3	<code>llm_service</code> validates shape, never repairs content	Stripping fabricated citations would mean the gate never fires and no audit record shows the model tried. Content trust belongs to the policy engine.	Fabricated proposals flow through to the gate on purpose
Sep 3	One retry, then template	A provider returning garbage twice is one to stop calling, not to keep paying.	A transient double-failure costs a template response rather than a retry storm
Sep 3	Provider exceptions logged by type , not message	Provider error text can carry request context; the type is also what actually distinguishes a timeout from an auth failure.	Less debugging detail in logs — the right trade for credential safety
Sep 3	<code>/score</code> accepts partial feature vectors	The model takes 392 features and no caller has them. LightGBM learns a NaN direction per split, so an absent feature is handled exactly as it was in training — where 76% of rows had no identity data. The response reports how many features were supplied, because a prediction from 4 features is not the same evidence as one from 392.	Callers can get a weak prediction without realising — mitigated by reporting the count
Sep 3	Calibrator classes live in <code>ml/calibrators.py</code> , never in a <code>__main__</code> script	Pickle records the qualified class path; anything defined in an executed script is only loadable by that script. See FAILURES 06.	One more module
Sep 3	No endpoint returns a bare proposal	<code>/disputes/draft</code> returns the draft <i>inside</i> the decision. An endpoint returning ungated LLM output would let a caller act on it, defeating the design.	<code>draft</code> and <code>analyze</code> return the same shape
Sep 3	<code>POST /batch/run</code> does not write audit records by default	Hitting it during a demo would append 5,013 records per press.	Opt in with <code>?write_audit=true</code>
Sep 4	Dashboard calls the live API rather than embedding numbers	Every verdict on screen is a real <code>POST /disputes/validate</code> . Hardcoding them would make the demo a slideshow, and the one thing worth showing is that the gate actually runs.	The page needs the server up; there is no static fallback

Date	Decision	Reason	Cost accepted
Sep 4	GET /metrics serves the committed evaluation artifacts, not recomputed values	The page is a view of what make all produced. If a number on screen disagrees with the repo, the repo is wrong — not the page.	Stale artifacts show stale numbers, which is the correct failure
Sep 4	Dashboard opens on the fabrication scenario	It is the one thing in the project that cannot be inferred from a metrics table.	—
Sep 4	Makefile resolves \$(PYTHON) rather than calling bare python/uvicorn	make api failed with "uvicorn: No such file or directory" for anyone without the venv activated — including a fresh clone, and make all is what a reviewer runs.	—
Sep 4	Evaluation JSON artifacts are committed; .pkl and prediction parquets are not	The JSONs are the numbers the README quotes, so a reviewer can check a claim without running anything. The binaries are regenerable and large.	Committed JSONs could go stale against the code — mitigated by the clean-clone reproduction check
Sep 4	The demo asserts its own outcomes and exits non-zero on drift	Recording and testing the same artifact means a policy change that would invalidate the video breaks the build instead of surfacing on playback.	The demo is slightly more code than a print-only script
Sep 4	A test asserts every policy rule has a demo scenario	Without it the demo silently falls behind the engine. It immediately caught two missing rules — including inside_cost_review_band, the most interesting one.	Adding a rule now requires adding a scenario
Sep 4	demo_04 repurposed from adapter_timeout to provider_unreachable	The Razorpay adapter was cut (Tier 3, never started), so the original case could not be demonstrated. The same property — an external dependency failing must not propagate — is shown against the LLM provider, a dependency that exists in this build and was verified failing against a live 429.	The recorded set no longer includes a Razorpay-specific failure
Sep 4	Keep one OpenAI-compatible client rather than a class per provider	Groq, Google and OpenAI all speak the same wire format, so LLM_BASE_URL + LLM_MODEL covers all three. A provider hierarchy would add indirection without capability — same argument as not using an agent framework.	Providers that are not OpenAI-compatible would need real work
Sep 4	Label the IEEE-CIS labelling methodology unverified rather than citing or deleting it	The host discussion thread needs a Kaggle account, which I could not open, and the description only reaches me through secondary sources. Citing it would have been a claim I had not checked; deleting it would leave a reader wondering why a fraud label is relevant to chargeback defence.	A paragraph that says "I could not verify this" — which is the honest state
Sep 4	Restructure so no result depends on the labelling methodology	The project needs a real externally-supplied label on real transactions, not a chargeback-derived one. Making that explicit removes the dependency instead of defending it.	None — the numbers were never derived from it
Sep 4	Demo gets --pause / --only / --list	The full run is ~180 lines and does not fit one terminal screen, so recording it meant scrolling on camera. Each scenario is 17–25 lines and fits.	A scenario registry instead of one linear function

Date	Decision	Reason	Cost accepted
Sep 4	Frontend redesigned light and restrained, dropping the dark neon palette	It read as a hacker dashboard rather than a product. For a payments panel the numbers should carry the page, not the chrome. One accent colour, used only on our own row; semantic colours only for the three actions, because those genuinely differ.	—
Sep 4	<code>decide()</code> returns the whole override chain, not just the rule that fired	A verdict says what happened; the chain says what was examined. An auditor asking "was the amount cap checked?" should not have to infer it. Recording is separate from deciding — <code>passed()</code> has no effect on control flow, so the ordering and early returns are unchanged.	Every decision carries 7 entries; the audit record grows
Sep 4	Frontend leads with the override chain rather than a verdict panel	A metrics dashboard is the generic part of any submission. The ordered deterministic gate is the part nobody without one can build a screen for.	The gate view is denser
Sep 4	Frontend rebuilt as a numbered decision record, not a dashboard	Two earlier passes read as a generic metrics dashboard — the same shape another buildathon entry was using. A compliance document is the least likely thing another team submits, and it argues the project's thesis (claim discipline) before a single number is read.	Serif/mono web fonts; denser and less immediately "app-like"
Sep 4	Limitations are §4 of the document, not a footer	In a report, stated assumptions are a numbered section a reader is expected to reach. Burying them would contradict everything else the page does.	—
Sep 4	Retry without JSON mode when a provider rejects <code>response_format</code>	It is the one call parameter that varies across OpenAI-compatible providers. A 400 naming it means "drop that parameter", not "give up" — the system prompt already demands a bare JSON object. Without this, switching provider could fail for a reason indistinguishable from the model being bad.	One extra attempt shape; a 429 or 401 is deliberately <i>not</i> treated this way
Sep 4	Label every dashboard proposal constructed , now that the live model is known not to fabricate	<code>make_llm-check</code> against Groq showed the model correctly refusing to invent a delivery record on the temptation probe. Leaving the fabrication scenario unlabelled would imply an LLM produced it.	The most striking demo case carries a caveat — correctly
Sep 4	Provider integration frozen after <code>make_llm-check</code> went green	Groq, <code>openai/gpt-oss-120b</code> , 3/3 schema-valid. Further provider work trades a finished submission for a nicer-to-have one on deadline day.	Only one provider is verified live; the swap path is tested but not exercised against a third
Sep 4	Adversarial cases live in their own file with a claim-limit docstring	Spreading them across <code>test_policy</code> and <code>test_failure_modes</code> made it possible to read a constructed attack as an observed model failure. One file, one header, one rule: these are engineering tests, not measurements.	—
Sep 4	<code>make_census</code> fails the build when any test asserts nothing	A raw pytest count flatters a suite. The check found three bare tests — one of them a fixture misnamed <code>test_*</code> that read as a test case to any reviewer skimming the file.	—

Date	Decision	Reason	Cost accepted
Sep 5	Static site precomputes decisions with the real engine rather than porting the gate to JavaScript	A JS reimplementation of the seven rules is a second implementation that can drift from the authoritative one — the exact argument already made for $p^*(A)$ having one implementation. Running <code>decide()</code> at build time keeps one engine and one truth.	Determinations on the hosted page are precomputed, and the banner says so
Sep 5	One source file serves both live and static modes	Forking <code>frontend/index.html</code> into a demo copy guarantees the two diverge. The page reads <code>window.__STATIC__</code> if a build injected it, else calls the API.	The page carries a small branch it does not need when served live
Sep 5	Hosted on GitHub Pages, not a deployed API	No server, no secrets in a third-party host, no cold start, nothing to break on submission day. A judge gets a link that works in one second and still works in a year.	The hosted page cannot demonstrate a live LLM call

17 Failure log

Written as each break happened, not reconstructed afterwards. This is the evidence for failure recovery, and several of these are more interesting than the features.

Failure 01 — Two spellings of "missing" in the baseline encoder

Problem. `tests/test_preprocessing.py::test_missing_categorical_becomes_its_own_level` failed: the fitted `OneHotEncoder` had learned levels `['T', 'None']` for `M1`, with no `__MISSING__` level, despite a `SimpleImputer(strategy="constant", fill_value="__MISSING__")` sitting directly upstream of it.

Cause. `SimpleImputer` matches `missing_values=np.nan`. The object columns come out of parquet holding Python `None`, which is not `np.nan`, so the imputer was a no-op and `None` survived into the encoder as a literal category. Training was unaffected — `None` is still a distinct level, and the metrics were valid — which is exactly why this would not have been noticed without the test.

Fix. Replaced the imputer with a module-level `fill_missing_categorical` using `df.where(df.notna(), "__MISSING__")`. `pandas.notna()` treats `None` and `NaN` identically, so both collapse onto one sentinel. Added `test_none_and_nan_encode_identically` as a regression guard.

Lesson. The bug was not a metrics bug, it was a **train/serve skew** bug, and it would have surfaced in the API rather than in training. A live `/score` request carries `np.nan`, which would not have matched the learned `None` level and would have fallen through `handle_unknown="ignore"` to an all-zero row — the model quietly ignoring a field it was trained on, with no error. Two representations of the same concept encoded two different ways is a class of bug that only a test comparing the two representations can catch. Assert on the *fitted artifact*, not just on the score. ---

Failure 02 — Isotonic calibrator was not monotone at float32 precision

Problem. `test_isotonic_is_non_decreasing` failed on 69 of 500 probe points, with the largest backward step at `-1.19e-07`.

Cause. Not a calibration bug — a dtype bug. Predictions are persisted to parquet as `float32` to keep the files small, and `IsotonicRegression` was fit and evaluated at that precision, so adjacent steps of the fitted step function differed by one ULP in the wrong direction.

Fix. Cast to `float64` inside `IsotonicCalibrator.fit/.predict`. Storage stays `float32`; the arithmetic does not.

Lesson. The magnitude was numerically irrelevant — no decision at any threshold changes by `1e-07`. Kept the fix anyway, because "a calibrator whose output can decrease as the model score increases" is not a sentence worth having to explain in a panel, and the cost of the fix was one cast. Storage precision and arithmetic precision are separate decisions, and letting the first silently set the second is how you get results that are *almost* right. ---

Failure 03 — The first cost curve said the product was unnecessary

Problem. The first working sweep returned a cost-optimal policy of "contest 99% of disputes", beating the trivial contest-everything rule by $\$37,433$ out of $\$394M$ — about 0.01%. Taken at face value: the model adds nothing and a merchant should just contest every chargeback.

Cause. Not the loss function — the population. I was pricing every held-out transaction as though it were a dispute, so the fraud base rate was 3.5%. Real dispute queues are nothing like that: a chargeback arriving is already strong evidence something went wrong. At 3.5% fraud, a median $\$6,028$ dispute and a $\$500$ representment cost, contesting is positive-EV almost regardless of score, so no threshold can have content. The degeneracy was an artefact of the population, not a property of the product.

Fix. Added a prior-shift sensitivity — re-calibrate the scores on the odds scale and re-weight the population to assumed dispute-queue fraud rates of 20/35/50/65%. The model's edge over contest-everything rises $\$2 \rightarrow \137 per dispute and the contest rate falls 99% $\rightarrow$ 60%. Production bands are now derived at a stated assumed queue rate (50%), not at the artefactual 3.5%. Also excluded disputes above the amount cap from the sweep, since the policy engine never decides those automatically.

Lesson. The arithmetic was correct and the answer was still meaningless, which is the most dangerous combination — nothing fails, no test goes red, and the number is simply about the wrong thing. Both the as-observed result and the shifted ones are reported: deleting the embarrassing one and keeping the flattering one would have been the easy move and the dishonest one. A base-rate-dependent conclusion needs its base rate stated as an assumption and swept, not inherited by accident from whatever data was lying around. ---

Failure 04 — The policy comparison measured a policy we do not ship

Problem. The README reported "ChargeShield beats the best no-model policy by $\$84$ per dispute" and called that "the value attributable to the ML". Re-running the comparison with the actual policy engine gave $-\$18$ — the shipped gate is slightly worse than the static amount rule across the queue.

Cause. `ml/evaluate.py:152` defined the row labelled `chargeshield` as `p_shift < bands["threshold"]` — a single global threshold on the score. The real gate uses amount-dependent bands, an evidence gate, a fabrication check, an amount cap and an economic floor. Worse, it *cannot* run in that module at all: `evaluate.py` operates on transactions, and transactions carry no `reason_code` and no evidence. So the row was not a degraded version of the policy engine, it was a different object with the product's name on it.

Fix. Renamed the row to `global_cost_threshold_rule` with an explicit note that it is not the shipped engine. Added `price_policies()` to `app/services/batch_runner.py`, which prices the four policies on the dispute queue using the actual `decide()` against real `isFraud` labels, plus a segment decomposition showing where the gate wins ($+\$69$ on actionable disputes) and where it pays for safety ($-\$83$ where evidence is missing). `tests/test_artifacts.py` now fails if a `chargeshield` key reappears in `evaluate.py`'s output, and asserts the batch row is produced by the real `decide()` rather than by a label.

Lesson. The most dangerous number is the flattering one that nobody re-derives. This survived three days and a commit message that quoted it, because every test checked that the arithmetic was *internally* consistent and none checked that the thing being priced was the thing being shipped. Naming is part of correctness: a variable called `chargeshield` that is not ChargeShield will be believed by everyone including its author. The corrected result is also a better story — "the gate costs $\$18$ /dispute and here is exactly what that buys" is defensible; $+\$84$ was not true. ---

Failure 05 — `evaluation/metrics.json` was not valid JSON

Problem. `evaluation/metrics.json` contained bare `NaN`, which no strict JSON parser accepts. The Day-5 frontend would have failed to load it.

Cause. `expected_cost()` was called with `t=np.nan` when a policy mask was supplied instead of a threshold, and `json.dumps` serialises `NaN` happily. Python's own `json.loads` also *reads* it back happily, so every test passed.

Fix. `threshold` is now `None` when a mask is given, and `tests/test_artifacts.py` parses every committed JSON artifact with `parse_constant` set to `raise` — the check Python does not do by default.

Lesson. Round-tripping through the same library proves nothing about interoperability. The test that mattered was the one that made Python behave like a stricter consumer. ---

Failure 06 — The calibrator artifact could only be loaded by the script that wrote it

Problem. `joblib.load("artifacts/calibrator.pkl")` raised `AttributeError: Can't get attribute 'PlattCalibrator' on <module '__main__'>` from every process except `python -m ml.calibrate` itself. The API could not load the calibrator at all, so `/score` and the demo would both have failed.

Cause. The calibrator classes were defined in `ml/calibrate.py`, which is executed as `__main__`. Pickle records a class by its qualified path, so the artifact referenced `__main__.PlattCalibrator` — a name that only resolves inside the process that happened to be running that file as a script.

Fix. Moved the classes to `ml/calibrators.py`, a module that is imported and never executed, so the pickled path is `ml.calibrators.PlattCalibrator` and resolves anywhere. Regenerated the artifact.

Lesson. Every test and every pipeline stage ran the calibrator inside the process that created it, so nothing exercised the load path from a different entry point. The bug was invisible until a second consumer existed, and it would have appeared for the first time on demo day. Pickling anything defined in a `__main__` script is a latent failure that waits for the second caller — and "we always run it the same way" is exactly the assumption that stops being true when you ship an API. ---

Failure 07 — `AuditLog`'s default path could not be redirected

Problem. API tests appended to the real `evidence/audit_log.jsonl`. The obvious fix — monkeypatching `audit_service.DEFAULT_PATH` — had no effect.

Cause. `def __init__(self, path: Path = DEFAULT_PATH)` binds the module constant at function-definition time, so patching the module attribute later changes nothing.

Fix. `path: Path | None = None`, resolved against `DEFAULT_PATH` inside the body.

Lesson. A default argument is evaluated once, at import. Using a module constant as one silently makes it un-overridable — which is a testing problem today and a deployment problem the first time the log needs to live somewhere else.

The pattern worth naming out loud: four of these seven produced no error and broke no test. Failure 01 trained fine and would have surfaced in production. Failure 03 was arithmetically correct and meaningless. Failure 04 was a flattering number nobody re-derived. Failure 06 was invisible until a second consumer existed. The bugs that cost real time were the silent ones.

18 Limitations and claim discipline

What can be claimed

- Precision, recall and PR-AUC on a temporally held-out slice of IEEE-CIS.
- A threshold that minimises expected loss *under stated assumptions*.
- Wasted representment effort, measured against real isFraud labels on anchor transactions.
- Policy-engine behaviour on constructed cases — evidence gating, fabrication blocking, deterministic override.
- Reproducibility: make all from a clean clone, 83s, bit-exact.

What cannot be claimed, and is not

- **Any win rate.** Real chargeback outcomes need merchant-side resolution labels that are not public.
- Money recovered, or reduction in anyone's real fraud losses.
- That IEEE-CIS is Razorpay data. It is US e-commerce from Vesta Corporation.
- That an LLM fabricated evidence. The live check found none; every fabrication case is constructed and labelled.

Stated assumptions — the attackable numbers

Assumption	Value	Why it cannot be measured
assumed_win_rate_if_legitimate	0.50 / 0.70 / 0.85	Pricing a forfeited winnable case requires knowing how often a representment succeeds. Not public, not in IEEE-CIS. Swept.
assumed_dispute_fraud_rate	0.50	IEEE-CIS has no dispute queue. Full sensitivity 3.5%-65% reported.
p_required_evidence_present	0.70	Evidence availability in the synthetic queue. Drives the human-review share, so that share is not a finding.
usd_to_inr	88.0	The dataset is USD. Rupee figures are a labelled overlay.

The IEEE-CIS labelling question

A description of how isFraud is defined circulates widely and is attributed to the competition host: a reported chargeback marks a transaction fraud, later transactions linked by card, account, email or billing address are marked fraud too, nothing reported within 120 days marks legitimate, and unreported fraud is therefore labelled legitimate.

That attribution is repeated as unverified. The host thread requires a Kaggle account; the description reaches this project only through secondary sources, and it does not cite what it has not read.

The project is built so it does not matter. Nothing requires the label to be chargeback-derived. It requires a real, externally-supplied binary label on real transactions, carried onto anchored disputes. If the methodology is something else, every number still stands; only the task's resemblance to a real chargeback queue changes — already a stated limitation. If the description *is* accurate it implies label noise in a known direction (unreported fraud labelled legitimate), so measured precision is pessimistic. No correction is claimed.

19 What the panel will ask

Ordered by how likely they are to come up. The short answer is what to say; the detail is what to say if pressed.

Q. Your policy engine costs more per dispute than a rule with no model in it. Why does it exist?

Because that number is the price of a safety property, and I can tell you exactly what it buys.

Across the queue it is Rs 18/dispute worse than the amount rule. On actionable disputes — evidence on file, under the cap — it is Rs 69 BETTER. The loss is entirely on disputes where evidence is missing, where it pays Rs 150 for a human rather than filing a representment it cannot substantiate. Human review costs Rs 84/dispute across the whole queue, more than the spread between all four policies. And the incomplete-evidence share is a parameter I set, not an observation, so the net moves with a dial rather than with the model.

Q. How do you know the LLM cannot cause an unsafe action?

Because the component that turns proposals into actions has no network client, no filesystem handle and no source of randomness.

decide() is a pure function taking thresholds as an argument. A test monkeypatches socket and open out from under it and asserts the gate still works — so the claim is enforced, not asserted. 43 adversarial cases attack it: fabricated citations, prompt injection in free text, malformed output, provider failure. One property holds across all of them: no bad proposal ever reaches CONTEST. Injection is inert by construction because the gate never reads the free-text fields.

Q. Did the LLM actually hallucinate evidence?

No. I tested for it and it did not.

The live check against Groq included a deliberate temptation — billing_proof present, shipping_proof absent — and the model correctly reported INSUFFICIENT rather than inventing a delivery record. So every fabrication case in the demo is a constructed proposal, labelled as such on the page and in the test file. The gate exists because a model may fabricate, not because this one did. I would rather say that than claim a hallucination I did not observe.

Q. Why should I believe your numbers?

Clone it and run make all. 83 seconds, and every figure matches to six decimal places.

Verified from a clean clone with a fresh venv: baseline PR-AUC 0.223152, LightGBM 0.543100, best iteration 610, Platt selected, decision-region ECE 0.047840, 5,013 disputes at queue rate 0.486535. All 18 headline README figures are checked against the committed artifacts programmatically.

Q. Your PR-AUC is 0.54. Is that good?

It is about 15x the base rate, and I am more interested in whether it is honest than whether it is high.

Prevalence is 3.5%. The split is chronological, so no future information leaks. My own red flag was PR-AUC > 0.90 — that would have meant hunting for leakage, not celebrating. The constrained linear baseline gets 0.223 on the same data with the same scoring code, and four of the tree's top eight features are exactly the columns the baseline excludes.

Q. Your calibration error is 0.048. Elsewhere you say 0.005. Which is it?

0.048. The 0.005 is the aggregate and it flatters the model.

92% of test rows score below 0.05, so an overall ECE is almost entirely a measurement of the region where no decision is made. I report the decision region, $p \geq 0.10$, where the gate acts. I found this because the reliability plot disagreed with the aggregate number, and I kept the worse figure as the headline.

Q. The dispute data is synthetic. Doesn't that invalidate everything?

It bounds what I claim, which is why one metric is real and the rest are behavioural.

Each dispute is anchored to a real held-out transaction and carries its real isFraud label. That makes wasted representment effort a measurement: 1,803 contested, 672 genuinely fraudulent, 37.3% against a 48.7% contest-all baseline. Everything else the dispute layer demonstrates is policy behaviour, and no win rate is claimed anywhere because that would need merchant-side resolution labels that are not public.

Q. Why is the review band amount-dependent rather than a single threshold?

Because the indifference point is analytically amount-dependent: $p^* = 1 - c/(w \cdot A)$.

A Rs 2,000 dispute and a Rs 20,000 dispute have genuinely different cut-offs — bands [0.536, 0.750] and [0.954, 0.975]. That is also why the amount cap exists: it is cost-derived, not an arbitrary rail. The demo shows the same $p=0.88$ producing HUMAN_REVIEW at Rs 6,070 and ACCEPT at Rs 2,000.

Q. What broke during the build?

Seven things, and four of them produced no error at all.

The instructive one: I reported that ChargeShield beat the best no-model policy by Rs 84/dispute. It did not — that row was measuring a global-threshold rule with the product's name on it. Priced with the actual gate it is Rs 18 worse. Every test passed the whole time, because they all checked the arithmetic was internally consistent and none checked that the thing being priced was the thing being shipped.

Q. Is this production-ready?

No, and I can list what is missing.

No Razorpay adapter — cut as Tier 3 and logged. No persistence beyond an append-only JSONL audit log. No authentication. One provider verified live. IEEE-CIS is US e-commerce, not Indian payments; the methodology transfers, the numbers do not. What is production-shaped is the separation: a pure, audited decision layer that no model can reach around.

Q. Why LightGBM and not a neural network?

Tabular data with 3.5% prevalence and 392 mostly-anonymised features.

Gradient boosting handles NaN natively — which matters when 76% of rows have no identity data — and splits on high-cardinality categoricals without embedding them. It also trains in 32 seconds, which meant I could spend the time on calibration, the cost model and the gate. Those moved the decision quality far more than another point of PR-AUC would.

Q. What would you do next with a week?

Calibrate within the decision region, and fix ProductCD W.

Decision-region ECE of 0.048 is mediocre and the rupee figures inherit it; calibrating on that region rather than the whole population would likely beat both current options. Separately, recall on ProductCD W is 0.213 against 0.703 elsewhere, and W is 79% of transactions and 86% of unrecovered fraud value. That is the single biggest modelling gap and I know exactly where it is.

20 Command reference

```
# setup – no target needs an activated venv
python3 -m venv .venv && make setup

# full reproducible path (83s), this is what a reviewer runs
make all          # data -> baseline -> train -> calibrate -> cost
                  #      -> evaluate -> link -> batch -> test

# individually
make data         # load, join, downcast, chronological split
make baseline     # logistic regression          -> test PR-AUC 0.223
make train        # LightGBM                    -> test PR-AUC 0.543
make calibrate    # Platt vs isotonic on val_fit/val_pick
make cost         # rupee cost curve + policy bands
make evaluate     # detection metrics, error slices, confound check
make link         # 5,013 disputes anchored to real transactions
make batch        # the gate over the whole queue

# demo and interface
make demo         # 8 self-asserting scenarios, no network needed
make demo ARGS="--pause"      # stop between scenarios – USE THIS TO RECORD
make demo ARGS="--only 2"     # just the fabrication case
make api          # http://127.0.0.1:8000

# verification
make test         # 214 cases
make census       # what is actually asserted, and what is not
make llm-check    # live provider probe; reports credential presence only

# provider swap – no code change
#   LLM_API_KEY=... LLM_BASE_URL=https://api.groq.com/openai/v1 LLM_MODEL=...
```

Regenerating this guide

```
python -m scripts.build_guide      # requires reportlab
```

Every number, decision and failure in this document is read from the repository at build time. It cannot drift from the artifacts it describes.